
How Reasoning Evolves from Post-Training Data: An Empirical Study Using Chess

Lucas Dionisopoulos¹ Nicklas Majamaki¹ Prithviraj Ammanabrolu¹

Abstract

We study how reasoning evolves in a language model – from supervised fine-tuning (SFT) to reinforcement learning (RL) – by analyzing how a set of theoretically-inspired datasets influences language model performance in chess. We find that fine-tuning a model to directly predict the best move leads to effective RL and the strongest downstream performance – however, the RL stage elicits *unfaithful* reasoning (reasoning inconsistent with the chosen move). Alternatively, training on multi-move trajectories yields comparable downstream performance with faithful reasoning and more stable RL. We analyze multiple qualitative and quantitative measures and highlight how these evolve from SFT through RL; we find several SFT-checkpoint metrics – spanning evaluation performance, hallucination rates, and reasoning quality – to be predictive of post-RL model performance. Finally, we ground our results with an experiment measuring *chess information density* in our custom datasets. We release models as well as training data, evaluations, and code that allowed us to surpass leading open-source reasoning models in chess with a 7B-parameter model. Code, models, and data are accessible at [lang-chess](#).

1. Introduction

What is required to train a language model to reason through RL? Several ingredients appear critical – a strong base model and a compatible domain are sensible starting points. *But what is a strong base model? And once you have a domain, how do you train it to reason effectively?*

¹Department of Computer Science and Engineering, University of California San Diego, San Diego, United States. Correspondence to: Lucas Dionisopoulos <lucasdionisopoulos@gmail.com>.

We seek to address these motivating questions by training a language model to reason in a verifiable, sequential decision process. Specifically, we choose chess as our focus because of several convenient elements: intrinsic difficulty for LLMs, established theory, favorable structure (episodic MDP), large datasets, and an efficient oracle (chess engines) for verifiable rewards and high-quality synthetic data generation. As a result, we can measure, in a controlled setting, how different training datasets influence our language model through SFT and how RL further evolves reasoning.

Language-based reasoning (OpenAI, 2024) has emerged as a technique to advance language model capabilities, although research remains largely confined to domains such as math and coding. Reasoning, often characterized as extending a model’s “chain-of-thought” behavior (Wei et al., 2022) using methods such as RL, benefits from these domains being clearly verifiable: the math is correct or the code passes all tests. This verifiable nature – combined with downstream applicability and skill transfer – has stimulated much research in these domains. As a result, reasoning models have achieved profound results across related benchmarks (OpenAI, 2025a; Anthropic, 2025; Google DeepMind, 2025), long-duration tasks (Kwa et al., 2025; METR, 2025), and earned an International Math Olympiad gold medal (DeepMind, 2025; OpenAI, 2025).

Although recent work explores reasoning in more subjective domains (Whitehouse et al., 2025) through techniques such as “LLM-as-a-judge” (Zheng et al., 2023), we focus here on those that are verifiable. The core verifiable domains – math and coding – benefit from years of continued research that has established a corpus of high-quality training data which produces strong out-of-the-box performance. While we benefit from stronger general base models, working on these explored domains would pose a challenge in isolating the influence of training interventions.

Chess thus emerges as attractive for studying reasoning. Language models have historically struggled with chess (Acher, 2023; Dynomight, 2024) and even state-of-the-art reasoning models still falter – often responding with illegal moves or simple blunders as seen in the Kaggle AI Chess Exhibition (Kaggle & Google DeepMind, 2025). Models underperform partly because chess data, while abundant,

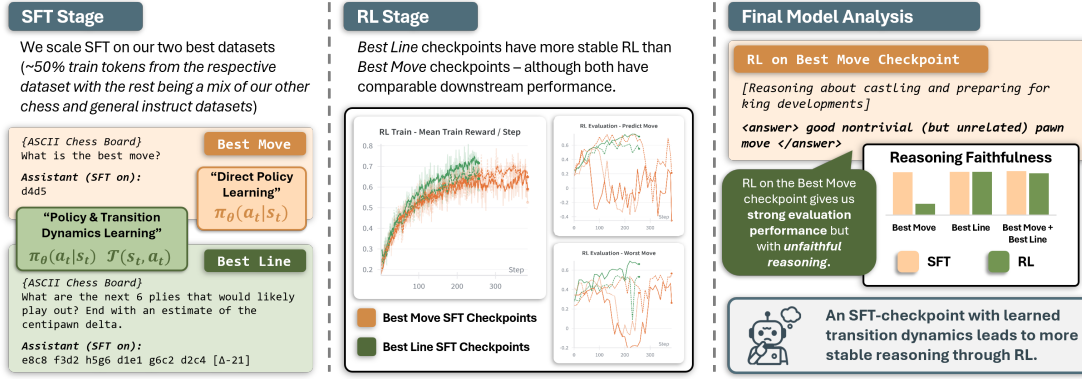


Figure 1. Following initial data inclusion experiments, we scaled SFT on our two best-performing datasets. Both resulted in comparably strong final evaluation performance, but training on optimal move trajectories (*Best Line*) led to more stable RL and faithful reasoning compared to training on single best moves (*Best Move*).

is rarely emphasized in pretraining; further, the game’s combinatorial structure makes generalization challenging. While this poses a difficulty, access to superhuman verification (in chess engines) provides an efficient method for verifiable rewards and synthetic data generation. All these aspects make chess a compelling testbed for reasoning.

Motivated by this setting, we train a 7B-parameter model on custom datasets using both SFT and RL to achieve performance surpassing gpt-oss-120b (OpenAI, 2025b) on several benchmarks. We focus our study on the following:

- *Q1*: How do different datasets (e.g., *programmatically generated, various synthetic generations*) impact downstream performance after SFT and RL?
- *Q2*: How does RL influence a model’s qualitative behaviors (e.g., *move quality distribution, reasoning strategies used, rate of hallucination*)?
- *Q3*: Which SFT-checkpoint metrics are predictive of final RL performance?

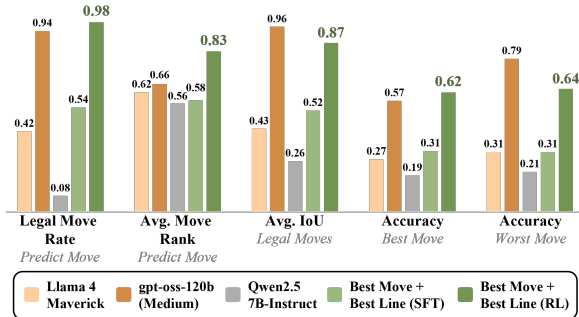


Figure 2. Performance of our best reasoning model trained from Qwen2.5 7B-Instruct across our evaluations. Trivial performance (i.e., random guessing) is 0.2 for the *Best Move* and *Worst Move* tasks. See Appendix B for example evaluation questions and Appendix D for full results.

We show that focused SFT on predicting a single best move (*Best Move*) leads to strong performance but *unfaithful* reasoning through RL; by contrast, training on multi-step move trajectories (*Best Line*) has more stable RL and faithful reasoning. We find that RL leads to fewer hallucinations and a substantial positive shift in move quality, and we see that several SFT-checkpoint metrics (both *qualitative* and *quantitative*) are predictive of final RL performance. Finally, we ground our results with an experiment measuring *chess information density* for each custom dataset.

2. Related work

2.1. Reasoning in language models

Reasoning in causal language models can be interpreted as self-guided search that makes a task more tractable. Consider a numerical math problem: effective reasoning should increase the probability of producing the correct number more than if the model had immediately predicted the final answer. Note that this reasoning need not be wholly interpretable – for example, it can exist in continuous space (Hao et al., 2024) or shift between languages (DeepSeek-AI et al., 2025) – what ultimately matters is that the intermediate steps are beneficial to the model. For this work, we focus on language-based reasoning.

The era of *reasoning models*, notably initiated with OpenAI’s release of o1 (OpenAI, 2024), builds upon much prior work in self-guided in-context adaptation. Models, when told to work “step by step” and write down intermediate results on a “scratchpad” (Nye et al., 2021), saw performance improvement on multi-step computations – this result was reinforced at scale and termed “chain-of-thought” in later work (Wei et al., 2022). Further, these reasoning traces can be used for iterated improvement through fine-tuning on successful generations as evidenced by STaR (Zelikman et al., 2022). Quiet-STaR (Zelikman et al., 2024)

extended this from fine-tuning by using a reinforcement learning policy-gradient update in REINFORCE (Williams, 1992) over tokens influenced by intermediate reasoning steps. This iterated bootstrapping using RL for policy-gradient updates has been the primary underlying method fueling the latest developments in reasoning models.

Following OpenAI’s release of o1, many leading systems began incorporating similar reasoning techniques to improve performance. Notably, DeepSeek-R1 (DeepSeek-AI et al., 2025) and Kimi k1.5 (Kimi Team et al., 2025) were among the first reasoning models to effectively approach state-of-the-art ability and publicize the training methods.

2.2. Reasoning through RL

While there exist several effective methods for training models to reason such as in-context prompting (Wei et al., 2022; Kojima et al., 2023), model distillation (DeepSeek-AI et al., 2025), or SFT on successful outputs (Zelikman et al., 2022; Yuan et al., 2023), we focus our attention on the setting of applying RL to improve model reasoning.

RL has been used as an effective tool to guide model behavior with Ouyang et al. (2022) inciting the viral *ChatGPT moment* that brought language models to public attention. Successful RL – regardless of domain – requires useful reward signal. For language models, these rewards can be generated using the following: rewards can be parsed and automatically calculated in verifiable tasks (Shao et al., 2024), determined directly through human judgment (Christiano et al., 2017), scored with a learned reward model (Ouyang et al., 2022), or elicited using a language model as a judge (Whitehouse et al., 2025). Rewards can be generated for the entire outcome or at intermediate steps (Lightman et al., 2023), and learned value functions can approximate credit-assignment at the token-level (Schulman et al., 2017) or reward can be indiscriminately applied over a full sequence (Shao et al., 2024). While this covers many methods in RL for language models, it is not exhaustive.

A common family of algorithms used in RL for language models is Proximal Policy Optimization (PPO) (Schulman et al., 2017) – an actor-critic method. Because actor-critic methods require learning a value function to address the credit-assignment problem (which can be computationally expensive and experience instability), new methods such as Group Relative Policy Optimization (GRPO) (Shao et al., 2024; DeepSeek-AI et al., 2025) have emerged to remove this learned value function requirement. GRPO has further evolved through variants such as Dr. GRPO (Liu et al., 2025), which removes sequence-level length normalization, and DAPO (Yu et al., 2025), which removes the KL penalty, increases the clipping bound to encourage exploration, and addresses length normalization issues observed in GRPO.

2.3. Chess engines

Computer scientists have developed grandmaster-level chess systems built on three notable techniques: 1) Classical search-based engines such as IBM Deep Blue (Campbell et al., 2002) or Stockfish that use a minimax-based search algorithm (commonly alpha-beta pruning), 2) neural search-based systems such as AlphaZero (Silver et al., 2017) and its open-source implementation in Leela Chess Zero that learn policy functions through RL self-play combined with Monte Carlo Tree Search, and 3) searchless neural systems such as Google DeepMind’s chess transformer that predicts a move directly from a board state (Ruoss et al., 2024). The respective 40/15 Elo scores of Stockfish and Leela Chess Zero as of April 19, 2026 are 3652 and 3444 (Computer Chess Rating Lists, 2026), and DeepMind’s chess transformer reached a Lichess blitz Elo of 2895 (Ruoss et al., 2024). While it is worth noting that recent versions of Stockfish use neural networks to estimate the value of board states – it still largely employs the same core algorithm used by classical search-based engines.

As discussed previously, language models struggle in the domain of chess. However, it is worth mentioning gpt-3.5-turbo-instruct which has an estimated Elo around 1700 (Acher, 2023). While this anomaly is interesting, this performance isn’t from language-based reasoning – rather it is direct next move prediction (i.e., only outputs the move to play). Google DeepMind’s chess transformer validated that a 270 million parameter transformer is capable of reaching grandmaster-level chess without search (*this was achieved by learning a value function, though a policy function was also tested*). As far as the authors are aware, no language model has achieved competition-level chess ability through language-guided reasoning and the best reasoning model in chess is OpenAI o3, which won the 2025 Kaggle AI Chess Exhibition (Kaggle & Google DeepMind, 2025).

3. Background

Our analysis is focused on the Qwen2.5 7B-Instruct model (Qwen et al., 2025). Given the baseline model has insufficient ability, we first conducted SFT prior to the RL stage. We began with a full set of data inclusion studies – from SFT to RL – to determine the most effective recipe before doing a final, scaled training run on our leading mix.

3.1. Board and move representation

For all training and evaluation we provide the board state in a visual ASCII-format. We ran preliminary tests on several board formats including Forsyth-Edwards Notation (FEN), FEN with space delimiters, and a visual ASCII-format. While these showed similar quantitative performance, we opted for the visual format following subjective qualitative

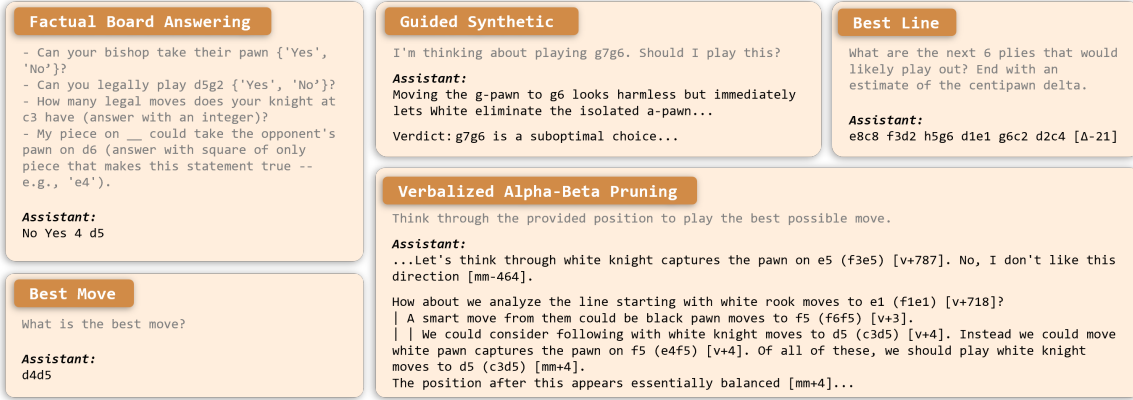


Figure 3. **Samples from our custom datasets.** The gray font represents an abbreviation of the core prompt – in all samples the model is trained with a verbose instructive prompt and provided with a board in our visual ASCII-format. Full samples included in Appendix C.

analysis. Appendix A provides examples of the considered board states and discusses tokenization limitations in each. Note that our board representation omits move repetitions due to dataset limitations – in competition chess, repetitions can be used as a termination condition. However, since none of our evaluations incorporates repetitions, we can view our representation as a Markov-complete state.

For move representation, we follow DeepMind’s chess transformer (Ruoss et al., 2024) and represent all moves in Universal Chess Interface (UCI) format (e.g., e1e2). This decision was made in lieu of formats such as Standard Algebraic Notation (SAN) which may be more commonly represented in training data – SAN has intricacies that could evoke errors avoidable by using UCI notation.

3.2. Evaluations and RL environment

We created four custom tasks that we use for evaluations and the RL training environment. The **Predict Move** task provides a board and asks the model to play the best move – no list of legal moves is provided. We measure both the ratio of legal moves generated as well as the move quality for legal moves provided. Move quality is measured as the normalized rank among legal moves ($\in [0, 1]$) as determined by a chess engine – where the best move is given a score of 1 and the worst move a score of 0. The **Best Move** and **Worst Move** tasks provide a board and a set of 5 moves. The task is to choose the best move (and worst move, respectively) of the candidate moves provided. For both tasks, candidate moves are sampled with a move quality threshold (determined by a chess engine) separating the correct answer from other candidates. Finally, the **Legal Moves** task asks the model to, for a given board and piece, list all legal moves that piece can make. Results are computed as intersection over union (IoU) versus the ground truth. We provide example questions in Appendix B.

3.3. Datasets

We created several theoretically-inspired datasets to study training dynamics from SFT to RL. Consider that chess can be represented as an MDP. At time t , there exists state $s_t \in \mathcal{S}$, playing a ply (i.e., half-move) constitutes an action $a_t \in \mathcal{A}(s_t)$, and an environment transition (i.e., opponent move) is a state transition $s_{t+1} \sim \mathcal{T}(s_t, a_t)$. Additionally, for each board state-action pair there is a reward $r_t = \mathcal{R}(s_t, a_t)$ which we can approximate using a shaped dense reward (centipawn delta, i.e., the change in an engine’s board evaluation measured in hundredths of a pawn) from a chess engine: $r_t = \gamma V_{\text{engine}}(s_{t+1}) - V_{\text{engine}}(s_t)$ with $\gamma = 1$. We will use this formulation to discuss motivation for several of our custom datasets.

We provide a brief description of each dataset and will further elaborate on data design and motivation within the context of experimental results in Section 4. We include detailed explanations of each dataset and full examples in Appendix C – abbreviated examples are included in Figure 3. Regarding our datasets, we organize them into the following four categories:

- **General Instruction Following:** Specifically, Magpie Llama 3.3 70B (Xu et al., 2024).
- **Rejection Sampling:** We generate outputs from Llama 4 Maverick (Meta AI, 2025) on our four evaluation types. We chose Llama 4 Maverick for qualitative and quantitative performance, retaining samples from the **Best Move** and **Worst Move** evaluations if correct and keeping outputs from the **Legal Moves** and **Predict Move** evaluations if above a threshold.
- **Guided Synthetic:** We prompt Llama 4 Maverick and gpt-oss-120b with a programmatically generated harness. Specifically, we provide a beginning board, 5 plies (the first ply being a move candidate and following plies being optimal play from a chess engine), and the ending

board state. The task is to generate an explanation of how the proposed candidate move will play out, ending with a final verdict for the proposed move.

• **Programmatically Generated Data:**

- **Factual Board Answering:** We build on top of a chess engine to generate simple question-answer (QA) pairs for a given board. These questions may ask if a move is legal, which square is threatening a specific piece, or how many legal moves a piece has. We combine multiple QA pairs for each sample.
- **Verbalized Alpha-Beta Pruning:** We use a custom program built upon Stockfish to sample moves, roll-out the line of play for each move (with branching and board values), and verbalize rollouts and minimax decisions in natural language. We explicitly build in tree search reasoning strategies and sample poor moves to verbalize the process of *pruning*, and we leverage a large, custom prompt bank to add diversity to natural language outputs.
- **Best Move:** Given a board, immediately predict the best move in UCI notation.
- **Best Line:** Given a board, predict the optimal line of play (4 – 6 plies) ending with the expected centipawn delta from playing this line.

3.4. Training environment

All SFT is conducted using LlamaFactory (Zheng et al., 2024) and all RL is conducted using veRL (Sheng et al., 2025). We utilize Dr. GRPO (Liu et al., 2025) for our RL optimization algorithm and employ the *Clip-Higher* strat-

egy with no KL divergence per Yu et al. (2025). A full list of hyperparameters is included in Appendix I.

4. Key findings

We ran a series of inclusion analyses to understand the efficacy of each data type and scaled our best-performing recipes. Figure 2 highlights the performance of our best reasoning model. We found the *Best Move* and *Best Line* datasets to be most effective – especially when lightly supplemented with our other, less effective datasets. Our scaled runs build on the *Best Move - All* and *Best Line - All* datasets that use this dataset diversity. The best final performance was achieved by first training Qwen2.5 7B-Instruct on 60 million tokens (*Best Move - All* data) followed by 60 million tokens (*Best Line - All* data). Appendix D provides further detail on our experiments.

4.1. Q1: How do different datasets impact downstream performance after SFT and RL?

Multitask training is beneficial for a fixed token budget, yielding higher move quality, less reward hacking, and a generally more robust model. This is shown by a comparison between *Rejection Sampling (Predict Move)* and *Rejection Sampling (All Evals)*: for the former we SFT and conduct RL on only the *Predict Move* task – for the latter we use all tasks. Our experimental results (Figure 5) led to these takeaways. We incorporate multitask training in all following experiments.

The most effective datasets were dense with difficult, high-quality tokens. We created three "dense" datasets



Figure 4. **RL training performance on our scaled SFT-checkpoints.** Left: Train reward and tokens per response (smoothed using an exponential moving average with decay factor 0.9). Right: Reward on the held-out evaluation set during training. The *Best Move* dataset, while having strong ending performance, experienced more unstable RL compared to scaled runs trained on *Best Line* data.

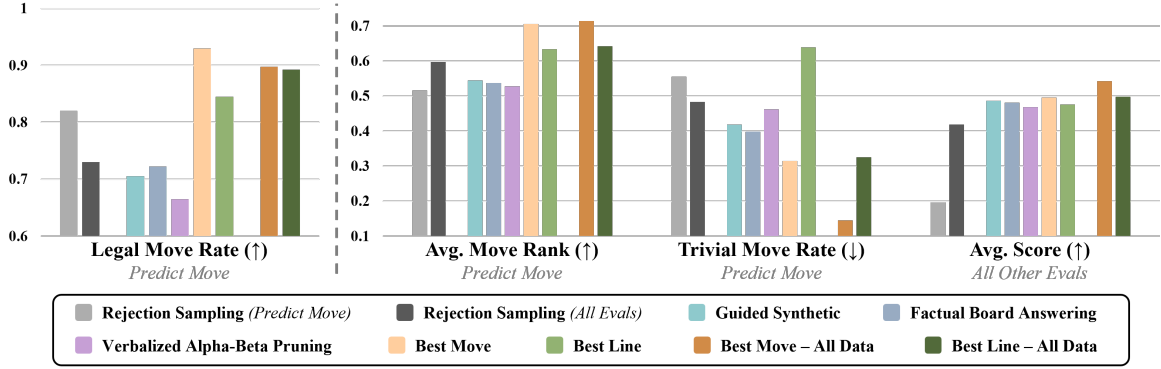


Figure 5. Results on evaluation tasks for final RL models from each data inclusion experiment. Within each metric we split results into three sections: (left) compares single vs. multitask, (middle) compares the targeted data inclusion experiments, (right) covers our data diversity experiments. Note that in all experiments we SFT on 15 million tokens and do RL on 8k samples. In the single task setting, RL only uses the *Predict Move* task; in all other settings the 8k samples are split evenly between our four task types. See Appendix D for detailed table results (including exact token distributions).

that lack prose: *Factual Board Answering*, *Best Move*, and *Best Line*. The intent was to force the model to embed complex board understanding and strategy in latent layers through immediate responses. These three largely performed the best, although *Factual Board Answering* was not materially better than our initial *Rejection Sampling* (*All Evals*) experiment. One explanation is that *Factual Board Answering* is more easily learned versus the other dense tasks. Consider *Best Move* – playing a strong move requires internalizing some level of board understanding and strategy, effectively encompassing *Factual Board Answering* and requiring a richer latent representation. We further discuss *chess information density* in Section 5 and provide a lens for interpreting dataset performance – but generally, our dense datasets performed best.

Dataset diversity is valuable. We see that the *All Data* experiments for *Best Line* and *Best Move* are superior to their focused counterparts (Figure 5). For these runs, we SFT on nearly all our data types – this comes despite mixed results on several of the individual inclusion analyses. Notably, the *Verbalized Alpha-Beta Pruning* experiment showed it was detrimental for training – we created this dataset as it is hallucination-free and includes rollouts and value functions (instilling $V(s_t)$ and $\mathcal{T}(s_t, a_t)$). However, it possesses a low density of high-quality tokens (moves and valuations) given the surrounding memorizable prompts. The *Guided Synthetic* dataset similarly produced subpar performance versus a *Rejection Sampling* baseline. Regardless, limited inclusion of all data was found to be beneficial.

Best Line had more stable RL training than Best Move. Figure 4 outlines RL training performance for our scaled runs – the models fine-tuned on *Best Line* data had more stable training dynamics. One reason may be that training on *Best Line* data – which includes multiple moves and ends with a valuation – allows the model to learn a world

model for chess (both a value function $V(s_t)$ and transition dynamics $\mathcal{T}(s_t, a_t)$). This is further supported by the coming discussion on *reasoning faithfulness* that reinforces this stability observation. Additionally, we find that models trained on *Best Line* perform better than our *Best Move* experiments on evaluations not trained on during RL – validating our observation that training on *Best Line* data leads to a more robust policy. Detail on the out-of-distribution evaluation is included in Appendix D.

4.2. Q2: How does RL influence a model’s qualitative behaviors?

Multi-step trajectory data led to the most faithful reasoners. We measure faithfulness by using gpt-oss-120b to judge alignment of final answers with reasoning traces. Our most faithful reasoner was achieved by training on the *Best Line* dataset which incorporates a rollout (in UCI) followed by a valuation (as a centipawn delta). This structure can be viewed as approximating n -step bootstrapping with $n = 2$ or 3 depending on ply depth. We can contrast this with the *Best Move* dataset which approximates a direct policy function (i.e., learning $\pi_\theta(a_t|s_t)$ via behavior cloning). Our multi-step trajectory checkpoints largely retained faithful reasoning through RL – on the other hand, the *Best Move* dataset became an *unfaithful* reasoner through RL, often displaying final, nontrivial answers that were disconnected from its reasoning trace. Figure 6 highlights reasoning faithfulness and Appendix H has further detail.

This is interesting as the unfaithful reasoner improves through RL without defaulting to trivial moves. Further, this improved ability is not explained by longer generations (Figure 4). One possible explanation we offer is the following: faithful reasoning from multi-step data may arise due to the model internalizing a chess world model (transition and value functions), whereas unfaithful reasoning may re-



Figure 6. **Reasoning faithfulness.** RL on the *Best Move* SFT-checkpoint induced *unfaithful* reasoning whereas checkpoints trained on multi-step data were more robust. Appendix H has further detail on our reasoning quality measurement.

sult from strong latent capability mixed with weak verbalized reasoning ability. Previous work (Turpin et al., 2023) has found that models may attempt to rationalize their answers in chain-of-thought unfaithfully if they are biased; in our case, the model may be attempting to rationalize the move it has "already decided".

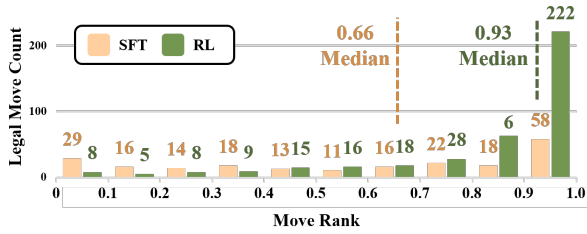


Figure 7. **Move quality distribution.** Our *Best Move + Best Line* scaled run saw a significant distribution shift after RL in its move quality on the *Predict Move* evaluation ($n = 400$). This shift is an improvement on both an absolute and relative basis, highlighting the efficacy of RL.

Regardless of reasoning faithfulness, **RL drove a substantial positive shift in move quality played** (Figure 7). Not only does RL improve the frequency of the best moves being played but it also decreases the frequency of low-quality moves on an absolute and relative basis. Additionally, **RL reduces hallucinations within reasoning traces** (Figure 8). This result is a side effect of rewarding correct answers as we do not incentivize factuality – we provide further detail on hallucinations in Appendix F and show that this result is shared across all inclusion experiments.

Lastly, we analyzed reasoning strategy usage at both the SFT and RL model stages. This follows prior work (Gandhi et al., 2025; Zeng et al., 2025) showing that effective reasoning models tended to utilize more reasoning strategies. We did not see clear trends in our analysis apart from our weaker models – specifically those more prone to reward hacking – almost exclusively reducing the usage of reasoning strategies through RL. In contrast, stronger models had mixed usage trends. We defer to Appendix G for detail.

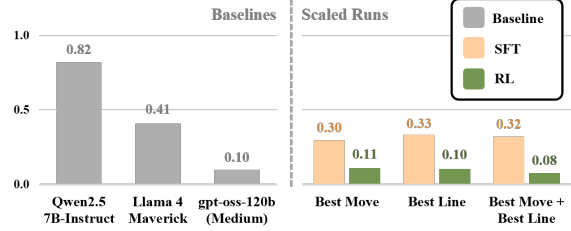


Figure 8. **Hallucination rate.** RL drove a meaningful decrease in hallucination rate as a side effect of simply maximizing reward on our evaluations. Appendix F has further detail on hallucinations.

4.3. Q3: Which SFT-checkpoint metrics are predictive of final RL performance?

Finally, we conducted a simple linear regression analysis comparing metrics from the SFT-checkpoint with the final RL model’s performance (average over all evaluations). Figure 9 highlights three SFT-checkpoint metrics that are statistically significant predictors of downstream performance.

Some of this is expected – an SFT model that scores higher on evaluations is likely better suited for RL. However, we also find qualitative signals to be highly predictive. Specifically, referenced move accuracy (*how little does the model hallucinate?*) and reasoning quality (*ask a language model to score how effective the reasoning is*) are statistically predictive of downstream performance. This shows that an effective SFT-checkpoint is one that is truthful (low hallucination rate), already an effective reasoner, and exhibits strong performance in the domain of focus.

5. Chess information density

Given our inclusion experiments hold the number of train tokens constant, we desire some measure for the information density of each dataset. While there are parallels between what we are terming *chess information density* and concepts in broader information theory, we instead take a first-principles approach and discuss related experiments, analyses, and qualitative results through a simple framework that offers intuitive takeaways.

There is no single, clear metric that quantifies *chess information density* – rather, there are several desirable attributes we would like in our datasets. First, the dataset should remain non-trivial to predict throughout training – we term this *predictive complexity*. Now, this itself is insufficient as both randomly generated data and text irrelevant to chess can evade triviality but be functionally useless for modeling chess. We must also consider *accuracy* (are the tokens correctly grounded to the task and board?) as well as *chess token density*. This latter concept hints at the idea that not all tokens are equal – some tokens may require board

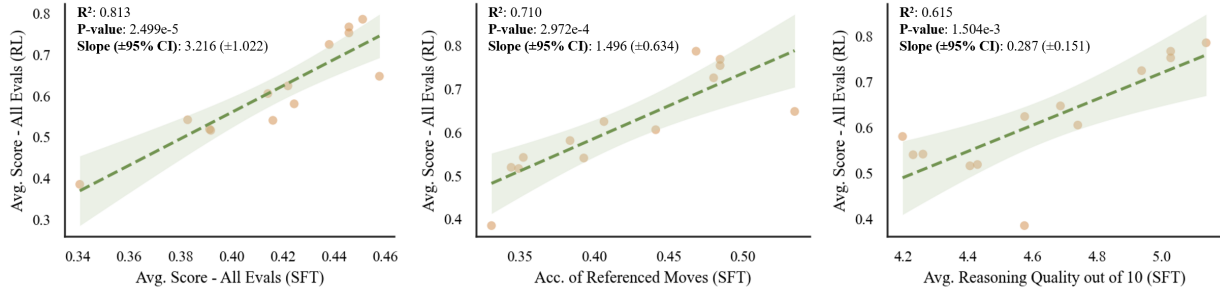


Figure 9. Linear regression comparing the final RL model (average score over all evaluations) with various metrics from its corresponding SFT-checkpoint. *Left:* Vs. average score over all evaluations. *Middle:* Vs. percent of moves referenced during reasoning trace that are legal (parsed by Llama 4 Maverick). *Right:* Vs. reasoning quality (mean over all reasoning quality metrics as judged by gpt-oss-120b). Shaded region represents the 95% confidence intervals.

understanding or chess strategy to effectively predict (e.g., in *Best Move* we ask to directly output the best move given a board). Compared to samples from the *Rejection Sampling* dataset – which have many natural language generic reasoning tokens – the *Best Move* data has higher *chess token density*.

To measure *predictive complexity*, we trained Qwen2.5 0.5B-Instruct (Qwen et al., 2025) on 4 million unique tokens for 2 epochs (total 8 million tokens). We hold out 5% of our dataset for validation and store probabilities associated with each validation token for further analysis.

Table 1. Fraction of validation tokens assigned probability >0.995 before and after SFT. Higher percentages indicate lower *predictive complexity* (e.g., simple task or memorizable phrases).

Dataset	Pre-SFT	Post-SFT
Rejection Sampling	12.69%	20.83%
Guided Synthetic	2.90%	9.77%
Factual Board Answering	0.04%	62.54%
Verbalized Alpha-Beta Pruning	4.31%	71.04%
Best Move	0.04%	28.61%
Best Line	0.00%	24.00%

Table 1 highlights the fraction of validation tokens that the model assigns a probability > 0.995 . Datasets with a high portion of trivial tokens can be interpreted as less complex – that is, the underlying patterns are more easily learned. Notably, we find *Verbalized Alpha-Beta Pruning* becomes trivial as the programmatic phrases used to verbalize search are quickly memorized – tokens that retain uncertainty often refer to a move decision or board valuation in their respective search branch. Additionally, the *Factual Board Answering* dataset has a high portion of trivial tokens. Upon further inspection (Table 2), we find that certain tasks become easy to predict while others remain more difficult. This is expected as not all tasks will be the same difficulty; however, it does have implications on dataset efficacy and benefits of scaling. We provide further information, including several visualizations, in Appendix E.

Table 2. Validation tokens assigned probability >0.995 before and after SFT for Factual Board Answering tasks. See Appendix C for descriptions of each task.

Question Type	Token Portion	Pre-SFT	Post-SFT
is_legal	13.59%	0.00%	24.85%
under_attack	12.43%	0.00%	30.46%
mobility	24.54%	0.01%	43.88%
cloze_capture	15.94%	0.14%	80.26%
is_check	4.95%	0.00%	85.16%
mat_adv_value	28.54%	0.05%	96.66%
Total	100.00%	0.04%	62.54%

Beyond *predictive complexity*, we must also consider *accuracy*. We have two synthetic datasets (*Rejection Sampling* and *Guided Synthetic*) that are prone to some amount of hallucination – the other datasets, as they are programmatically generated, avoid this problem. While we find the hallucination rate to be higher on synthetic datasets vs. programmatically generated datasets (Appendix F), there are several confounding factors at play. At best, dataset *accuracy* is one of many factors that play into dataset quality.

The final measure to consider is *chess token density* and is more qualitative. Of our datasets, *Verbalized Alpha-Beta Pruning* and *Rejection Sampling* have the highest portion of generic natural language. *Guided Synthetic*, while still natural language, is prompted to produce a succinct verdict following prior reasoning – as a result, *Guided Synthetic* has lower triviality compared to *Rejection Sampling* (Table 1). Our last three datasets we consider to be dense as they lack prose: *Factual Board Answering*, *Best Move*, and *Best Line*. All three were designed for *chess token density*; however, we see in Table 1 there are discrepancies in *predictive complexity* across the three that align with our observed results. Intuitively, in *Factual Board Answering* we ask the model to answer various questions requiring board understanding. Compare this to *Best Move* where, to play an effective move, much of this board understanding needs to

be internalized and considered. Now, compare *Best Move* with *Best Line* – *Best Line* requires tracking board state and modeling competing lines from players. Our dense datasets – those with higher *chess token density* – tend to perform better; among the dense datasets, those with higher *predictive complexity* tended to perform better as well.

We believe our broader results can be partially explained by these measures of *chess information density*. However, we also see that dataset diversity is beneficial (*Best Move* - *All* outperforming *Best Move*), and data such as *Guided Synthetic* – which should be superior to *Rejection Sampling* – does not outperform. Thus, these proximal measurements are a useful perspective but do not represent the full picture.

6. Limitations & further discussion

To begin, the intention of this work has always been to study general reasoning properties in language models. Thus, while the final evaluation of our model is a welcome result, we focused much of our effort on understanding the qualities and development of reasoning; this means that there are many methods we believe could further improve a chess reasoning model beyond our final RL model. For example, in full-game play our final RL model had poor performance against OpenAI o3. We suspect some level of distribution mismatch: training emphasized mid- and late-game positions to reduce trivial moves, which likely degraded opening play and hurt head-to-head results versus an opponent with stronger opening theory.

We have also identified several unexplored techniques that could increase performance in our final RL model. We minimally experimented with reward function tuning in our RL environment and expect focused effort could improve performance – particularly on the *Predict Move* task. Further, incorporating multi-turn RL and chess puzzles to more closely mimic a full chess game would likely yield strong improvements.

Regarding model choice, we acknowledge that our experiments were confined to Qwen2.5 7B-Instruct – while it would have been valuable to replicate on distinct base models, due to constraints this was not pursued. Additionally, we chose gpt-oss-120b as our comparator because, in tests against Kimi K2 and DeepSeek-R1-0528, it showed state-of-the-art open-source performance and was more convenient to run with our available resources.

Finally, we believe that there is a promising direction in training on environment dynamics to improve general reasoning abilities that is beyond the scope of this paper. Recent works such as CodeI/O (Li et al., 2025) and Absolute Zero (Zhao et al., 2025) show that training a model to predict program outputs from both code and inputs produces strong reasoners. We believe this can be pursued in other

sequential domains as well – for example, dialogue or embodied agentic tasks – where the model can be trained on both actions and responses.

7. Conclusion

We conduct a detailed study of how various custom datasets influence training dynamics through SFT and RL in the domain of chess. Our analysis highlights that training to predict the best move directly produces strong downstream performance but comes with *unfaithful* reasoning. Instead, training on multi-move trajectories delivers similar performance with faithful reasoning. We find that RL leads to fewer hallucinations and a substantial positive shift in move quality, and we see that several SFT-checkpoint metrics (both *qualitative* and *quantitative*) are predictive of final RL performance. Finally, we ground our results with an analysis of *chess information density*. We publish our code and data as well as scaled SFT-checkpoints and RL models.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Acher, M. Debunking the chessboard: Confronting gpts against chess engines to estimate elo ratings and assess legal move abilities. <https://blog.mathieuacher.com/GPTsChessEloRatingLegalMoves/>, September 2023. Blog post by Professor Mathieu Acher on variability in GPT chess performance.
- Anthropic. Introducing claude 4, May 2025. URL <https://www.anthropic.com/news/claude-4>. Accessed: 2025-08-15.
- Campbell, M., Hoane, A., and hsiung Hsu, F. Deep blue. *Artificial Intelligence*, 134 (1):57–83, 2002. ISSN 0004-3702. doi: [https://doi.org/10.1016/S0004-3702\(01\)00129-1](https://doi.org/10.1016/S0004-3702(01)00129-1). URL <https://www.sciencedirect.com/science/article/pii/S0004370201001291>.
- Christiano, P., Leike, J., Brown, T. B., Martic, M., Legg, S., and Amodei, D. Deep reinforcement learning from human preferences, 2017. URL <https://arxiv.org/abs/1706.03741>.
- Computer Chess Rating Lists. Ccrl 40/15 rating list, 2026. URL <https://computerchess.org.uk/ccrl/4040/index.html>. Accessed: 2026-04-23.

- DeepMind. Advanced version of gemini with deep think officially achieves gold-medal standard at the international mathematical olympiad, July 2025. Accessed: 2025-08-15.
- DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Ding, H., Xin, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Wang, J., Chen, J., Yuan, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Ye, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W., Zhao, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Xu, Y., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Dynomight. Something weird is happening with llms and chess. <https://dynomight.net/chess/>, November 2024. Blog post; exact publication date unspecified, referenced as November 14, 2024.
- Gandhi, K., Chakravarthy, A., Singh, A., Lile, N., and Goodman, N. D. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective stars, 2025. URL <https://arxiv.org/abs/2503.01307>.
- Google DeepMind. Gemini model thinking updates: Gemini 2.5 thinking, March 2025. URL <https://blog.google/technology/google-deepmind/gemini-model-thinking->
- [updates-march-2025/#gemini-2-5-thinking](https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/#gemini-2-5-thinking). Accessed: 2025-08-15.
- Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J., and Tian, Y. Training large language models to reason in a continuous latent space, 2024. URL <https://arxiv.org/abs/2412.06769>.
- Kaggle and Google DeepMind. Introducing kaggle game arena. <https://www.kaggle.com/blog/introducing-game-arena>, August 2025. Blog announcement of a new benchmarking platform where AI models compete in strategic games.
- Kimi Team, Du, A., Gao, B., Xing, B., Jiang, C., Chen, C., Li, C., Xiao, C., Du, C., Liao, C., et al. Kimi k1.5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. Large language models are zero-shot reasoners, 2023. URL <https://arxiv.org/abs/2205.11916>.
- Kwa, T., West, B., Becker, J., Deng, A., Garcia, K., Hasin, M., Jawhar, S., Kinniment, M., Rush, N., Arx, S. V., Bloom, R., Broadley, T., Du, H., Goodrich, B., Jurkovic, N., Miles, L. H., Nix, S., Lin, T., Parikh, N., Rein, D., Sato, L. J. K., Wijk, H., Ziegler, D. M., Barnes, E., and Chan, L. Measuring ai ability to complete long tasks, 2025. URL <https://arxiv.org/abs/2503.14499>.
- Li, J., Guo, D., Yang, D., Xu, R., Wu, Y., and He, J. Codei/o: Condensing reasoning patterns via code input-output prediction, 2025. URL <https://arxiv.org/abs/2502.07316>.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>.
- Liu, Z., Chen, C., Li, W., Qi, P., Pang, T., Du, C., Lee, W. S., and Lin, M. Understanding r1-zero-like training: A critical perspective, 2025. URL <https://arxiv.org/abs/2503.20783>.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Meta AI. The llama 4 herd: The beginning of a new era of natively multimodal ai innovation, April 2025. URL <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>. Accessed: 2025-08-13.

- METR. Details about metr’s evaluation of openai gpt-5. <https://metr.github.io/autonomy-evals-guide/gpt-5-report/>, August 2025. Accessed: 2025-08-15.
- Mészáros, A., Reizinger, P., and Huszár, F. Out-of-distribution tests reveal compositionality in chess transformers, 2025. URL <https://arxiv.org/abs/2510.20783>.
- Nye, M., Andreassen, A. J., Gur-Ari, G., Michalewski, H., Austin, J., Bieber, D., Dohan, D., Lewkowycz, A., Bosma, M., Luan, D., Sutton, C., and Odena, A. Show your work: Scratchpads for intermediate computation with language models, 2021. URL <https://arxiv.org/abs/2112.00114>.
- OpenAI. Learning to reason with llms. <https://openai.com/index/learning-to-reason-with-llms/>, September 2024. Blog release, September 12 2024.
- OpenAI. Introducing gpt-5, August 2025a. URL <https://openai.com/index/introducing-gpt-5/>. Accessed: 2025-08-15.
- OpenAI. Introducing gpt-oss, August 2025b. URL <https://openai.com/index/introducing-gpt-oss/>. Accessed: 2025-08-13.
- OpenAI. We achieved gold medal-level performance on the 2025 International Mathematical Olympiad with a general-purpose reasoning LLM! <https://x.com/OpenAI/status/1946594928945148246>, July 2025. Tweet.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback, 2022. URL <https://arxiv.org/abs/2203.02155>.
- Panickssery, A., Bowman, S. R., and Feng, S. Llm evaluators recognize and favor their own generations, 2024. URL <https://arxiv.org/abs/2404.13076>.
- Qwen, :, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Ruoss, A., Delétang, G., Medapati, S., Grau-Moya, J., Wenliang, L. K., Catt, E., Reid, J., and Genewein, T. Grandmaster-level chess without search. *CoRR*, 2024.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys 25, pp. 12791297. ACM, March 2025. doi: 10.1145/3689031.3696075. URL <http://dx.doi.org/10.1145/3689031.3696075>.
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017. URL <https://arxiv.org/abs/1712.01815>.
- Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edition, 2018. URL <http://incompleteideas.net/book/the-book-2nd.html>.
- Turpin, M., Michael, J., Perez, E., and Bowman, S. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 74952–74965. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/ed3fea9033a80fea1376299fa7863f4a-Paper-Conference.pdf.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837, 2022.
- Whitehouse, C., Wang, T., Yu, P., Li, X., Weston, J., Kulikov, I., and Saha, S. J1: Incentivizing thinking in llm-as-a-judge via reinforcement learning, 2025. URL <https://arxiv.org/abs/2505.10320>.

- Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992. doi: 10.1007/BF00992696.
- Xu, Z., Jiang, F., Niu, L., Deng, Y., Poovendran, R., Choi, Y., and Lin, B. Y. Magpie: Alignment data synthesis from scratch by prompting aligned llms with nothing, 2024. URL <https://arxiv.org/abs/2406.08464>.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Dai, W., Fan, T., Liu, G., Liu, L., Liu, X., Lin, H., Lin, Z., Ma, B., Sheng, G., Tong, Y., Zhang, C., Zhang, M., Zhang, W., Zhu, H., Zhu, J., Chen, J., Chen, J., Wang, C., Yu, H., Song, Y., Wei, X., Zhou, H., Liu, J., Ma, W.-Y., Zhang, Y.-Q., Yan, L., Qiao, M., Wu, Y., and Wang, M. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- Yuan, Z., Yuan, H., Li, C., Dong, G., Lu, K., Tan, C., Zhou, C., and Zhou, J. Scaling relationship on learning mathematical reasoning with large language models, 2023. URL <https://arxiv.org/abs/2308.01825>.
- Zelikman, E., Wu, Y., Mu, J., and Goodman, N. D. Star: Bootstrapping reasoning with reasoning, 2022. URL <https://arxiv.org/abs/2203.14465>.
- Zelikman, E., Harik, G., Shao, Y., Jayasiri, V., Haber, N., and Goodman, N. D. Quiet-star: Language models can teach themselves to think before speaking, 2024. URL <https://arxiv.org/abs/2403.09629>.
- Zeng, W., Huang, Y., Liu, Q., Liu, W., He, K., Ma, Z., and He, J. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild, 2025. URL <https://arxiv.org/abs/2503.18892>.
- Zhao, A., Wu, Y., Yue, Y., Wu, T., Xu, Q., Yue, Y., Lin, M., Wang, S., Wu, Q., Zheng, Z., and Huang, G. Absolute zero: Reinforced self-play reasoning with zero data, 2025. URL <https://arxiv.org/abs/2505.03335>.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL <https://arxiv.org/abs/2306.05685>.
- Zheng, Y., Zhang, R., Zhang, J., Ye, Y., Luo, Z., Feng, Z., and Ma, Y. Llamafactory: Unified efficient fine-tuning of 100+ language models, 2024. URL <https://arxiv.org/abs/2403.13372>.

A. Board format

We tested various board representation formats – the three formats shown in Figure 10 had similar initial evaluation performance on a baseline Qwen2.5 model. However, upon qualitative analysis, Visual (ASCII) format was ultimately chosen. Additional rationale and comments on each are listed below:

- **FEN:** The tokenizer combines specific characters (e.g., \n, RK, PPP) and this may limit generalization. Additionally, uneven tokenization across rows may hinder spatial understanding.
- **Spaced FEN:** While this format resolves combined character issues, there is now an inconsistent representation of spaces: ' 2' is two tokens while ' p' is one token. This may present issues in downstream spatial understanding.
- **Visual (ASCII):** Ultimately chosen because it alleviates concerns mentioned in Spaced FEN.

Note: We recommend that future practitioners alter the Visual (ASCII) format. Qwen-series (2 and 3) and Llama-series (3 and 4) tokenizers treat ' . \n' as a single token with ' p\n' as two tokens – this can be fixed by including a space before each newline. This inconsistency was discovered late in training and thus not integrated into our project. We include an updated `uniform_visual` board format in our [released code](#) that improves upon Visual (ASCII).

Forsyth-Edwards Notation (FEN)

Spaced FEN

Note: Tokenization shown on two rows for visual purposes.

Visual (ASCII)

Visual (ASCII) Format

```

8| r . . q . . k .
7| n p p . . . p .
6| p . . . b N . p
5| . . . . p . . Q
4| . . . . P . . .
3| . . . P . . . .
2| P P P . . . P P
1| R . . . . R K .

  A B C D E F G H
    
```

- It is Black's turn to move.
- No castling rights available.
- No en passant target square.
- Halfmove clock: 0
- Fullmove number: 16

Figure 10. Visualized tokenization of three candidate board formats using the Qwen2.5 tokenizer.

C. Dataset types and samples

We now outline more detail on the format and creation of our datasets. Full datasets – including associated statistics – are available on [Hugging Face](#).

For **General Text Alignment**, we use samples from the Magpie (Xu et al., 2024) Llama 3.3 70B dataset as a means of regularization. Given that a significant portion of the SFT data is dense in chess notation, we felt it valuable to maintain the instruction-following abilities of our trained model to prevent performance collapse.

Our **Rejection Sampling** data comprises four separate tasks, *Best Move*, *Worst Move*, *Predict Move*, and *Legal Moves*. We chose to use Llama 4 Maverick to generate synthetic rejection-sampled data as it offered strong performance, was efficient to run on our hardware, and offered more natural-sounding reasoning compared to other reasoning models. Our motivation for this dataset was to instill correct reasoning format and bake in ability from the teacher model given Qwen2.5 7B-Instruct had incredibly poor base performance. This motivation follows Yuan et al. (2023) which found rejection sampling fine-tuning to be highly effective for improving weaker models.

For each evaluation set, we sampled outputs from Llama 4 Maverick (Meta AI, 2025) – retaining answers that were correct on *Best Move* and *Worst Move* and keeping outputs on *Predict Move* and *Legal Moves* if the answers met a threshold. This data was further sampled to ensure a desirable distribution regarding move count and piece types, as rejection sampling inherently shifts the distribution towards board states that the data-generating model performs well on.

Rejection Sampling (Predict Move) Sample:

Assistant (SFT):

<think>

First, let's analyze the given position:

The current state of the board is:

```
8| r n . q . b n r
7| p . . . p k . p
6| . p . . . . p .
5| . . p . p . . .
4| . . . . . . .
3| . . . P . . P .
2| P P P . . P . P
1| R N B . K . . .
```

It's Black's turn. We need to assess the position carefully.

The first observation is that Black's king is somewhat exposed on e7, and there's no clear indication that it's going to castle soon since the kingside pieces haven't moved much. However, we also see that White's king is still in the center and hasn't castled yet, although it has the right to castle queenside.

...

A straightforward plan involves moving the f8 rook into play or preparing to develop other pieces effectively.

Looking closely at possible moves, `g7g6` appears too passive without directly contributing to rapid development or attacking play. Let's evaluate more active options like knight moves or pushing central pawns when feasible.

However, upon closer inspection, I notice that `b8c6` develops a piece, supports potential central pushes (`d7d5`), and doesn't overly expose Black. It's a solid developmental move.

Another consideration is assessing whether there are direct attacks or tactical shots available. For instance, checking if any of White's under-developed status leaves them open to tactics.

Upon deeper review, I believe `b8c6` fits well because it develops a piece harmoniously and gets ready for further actions such as controlling the center or supporting potential breaks.

</think>

<answer> b8c6 </answer>

Figure 12. A *Rejection Sampling* example. The full response in the sample is shortened for space. Note that a drawback of this dataset is that it is prone to hallucinations as shown in the provided sample.

To construct our **Guided Synthetic** data, we generate synthetic data by using a sufficiently strong teacher model to verbalize outcomes of a move. A teacher model (Llama 4 Maverick or OpenAI gpt-oss-120b) is provided with a beginning board state, line of up to 5 total plies (where all plies following the first move are the top suggested chess-engine move), and an ending board state. The model is tasked with verbalizing the merit of the proposed ply – first providing logic then a verdict on the candidate move's quality (given how the board would develop).

In the MDP setting, this can be interpreted as verbalizing n -step bootstrapping (Sutton & Barto, 2018) with $n = 3$ (given 5 plies yields 3 player actions). This is due to a verbalized transition function in the logic (outlining how the board will

develop – $\mathcal{T}(s_t, a_t)$) combined with a verbalized value function ($V(s_t)$) in the verdict. The intention of the harness is multifaceted – ideally it should reduce hallucination rate, increase *chess information density*, and also guide the model to generate this verbalized n -step bootstrapping format with meaningful rationales and verdicts.

Guided Synthetic Sample:

User:

Below is a board in a game I'm currently playing.

```
8| r . b . . r k .
7| . p . . b p p p
6| . q p p . n . .
5| . . . P p . . .
4| p . P . P . . .
3| . P . . . N . P
2| P . B N . . P .
1| . R . Q . R . K

  A B C D E F G H
```

- It is Black's turn to move.
- No castling rights available.
- No en passant target square.
- Halfmove clock: 0
- Fullmove number: 18

I'm thinking about playing g7g6. Do you think I should play this move?

Assistant (SFT):

Moving the g-pawn to g6 looks harmless but immediately lets White eliminate the isolated a-pawn with b3xa4. Black recovers only by shuffling the queen to a7 and the c8-bishop to d7, gaining modest development while staying a pawn down. The pawn push also weakens the dark-square complex around the king (g6/f5) and does nothing to address White's central pressure on d5/e4. Overall the line leaves Black with less material and no clear compensation – a clear downside.

Verdict: g7g6 is a suboptimal choice; avoid it in favor of a move that keeps the a-pawn or improves piece activity without conceding material.

Figure 13. Sample of the *Guided Synthetic* data. Note that while the teacher model is provided with a line and centipawn difference, the teacher model is still prone to hallucination (it cites a material imbalance – this is not true). Additionally, despite prompting the teacher to use UCI notation, many examples still use SAN due to teacher model bias.

The **Verbalized Alpha-Beta Pruning** dataset is an entirely programmatically generated dataset built on top of Stockfish. This uses a program that does the following:

- For a given board, apply softmax with temperature sampling to choose initial move candidates (based on Stockfish move valuations).
- For each candidate, recursively build a tree that employs a similar softmax-based sampling algorithm.
- The recursion ends when a depth limit is reached, a max number of nodes are explored for this move, or a move is "written off" (below a quality threshold compared to other lines).
- Upon creation of the full move tree, each tree is verbalized using a separate algorithm that samples phrases from a large prompt bank to retain language diversity.
- The final winner (chosen via a minimax-based decision) is verbalized and used as the final answer.

We chose to include board valuations as well as minimax scores at decision nodes to instill a sort of value function ($V(s_t)$) in the model. Note that this value function is the absolute centipawn score – not a delta; we chose this to allow for verbalized minimax decision making. Further, we instill several tree search strategies into the structure of the data – for example branching search, pruning, and minimax decision making.

The **Factual Board Answering** dataset generates question-answer pairs about board states and combines them to ask multiple questions about the same board. We have the following question types:

- **Is Legal** (`is_legal`): Asks whether a particular move is legal. Note that all candidates are legal motion patterns (e.g.,

Verbalized Alpha-Beta Pruning Sample:
Assistant (SFT):

Interesting, let's consider some moves here.

<think> Ok, what if we played white queen moves to e1 (d1e1) [v+605]?

| From here they'll likely consider black bishop moves to g7 (f8g7) [v-66].

| | Ok, then we could play white pawn moves to a4 (a3a4) [v-56].

| | They could also consider black pawn moves to h5 (h6h5) [v-71].

| | We could consider following with white knight moves to d5 (c3d5) [v-38].

| If the opponent played optimally, of the moves f8g7 and h6h5 they would likely choose black bishop moves to g7 (f8g7) [mm-56] as this gives them the best position. This position seems slightly better for the opponent [mm-56].

Let's think through white knight captures the pawn on e5 (f3e5) [v+787]. No, I don't like this direction [mm-464].

How about we analyze the line starting with white rook moves to e1 (f1e1) [v+718]?

| A smart move from them could be black pawn moves to f5 (f6f5) [v+3].

| | We could consider following with white knight moves to d5 (c3d5) [v+4]. Instead we could move white pawn captures the pawn on f5 (e4f5) [v+4]. Of all of these, we should play white knight moves to d5 (c3d5) [mm+4].

The position after this appears essentially balanced [mm+4].

Our best move given how these would play out against optimal opponent play would be white rook moves to e1 (f1e1). [mm+4] </think>

<answer> f1e1 </answer>

Figure 14. Sample of Verbalized Alpha-Beta Pruning. This sample highlights branching, minimax decision making, and an instance of pruning.

bishop proposal will be something diagonal from the source square). We upweight difficult cases such as providing a legal move (i.e., unblocked) but the king is in check (thus illegal).

- **Under Attack** (under_attack): We ask whether a particular piece type could take another particular piece type (e.g., *can your bishop take their pawn?*).
- **Mobility** (mobility): For a given piece, how many legal moves does it have (as an integer)?
- **Cloze Capture** (cloze_capture): Here, we ask to fill in the only square that satisfies a statement such as: *my piece on __ can take the opponent's rook on b8*.
- **Is Check** (is_check): We simply ask if a side's king is in check.
- **Material Advantage Value** (mat_adv_val): We provide a list of piece values (*canonical valuations*) and ask to compute the material value differential from one side's perspective.

This dataset has the intent of training a model to explicitly learn piece and board dynamics in its latent space with the hope that this latent ability will translate to downstream reasoning performance. Note that we took great care in balancing the distribution of answers and hard negative mining within each question type – please see the [Hugging Face Dataset](#) for more information.

The **Best Move** dataset asks the model to, given a board state, predict the best move directly with no chain of thought. This can be interpreted as learning a policy function ($\pi_{\theta}(a_t|s_t)$) via behavior cloning where the best move is suggested by a chess engine. One drawback of training on this data is that it can lead to very slow fine-tuning given each sample has 4 trainable tokens.

Our final dataset – **Best Line** – is similar to *Best Move* except that it asks the model to provide the line of optimal play (between 4 and 6 plies, chosen randomly to avoid a rigid structure), ending with a final estimate of the centipawn delta from this line. This is similar to *Guided Synthetic* and can be interpreted as verbalized n -step bootstrapping with $n = 2$ or 3 dependent on the number of total plies.

Our intent is that this will instill a form of a world model through learning both transition dynamics and a value function. It can also be viewed as an extension of *Best Move* that condenses multiple optimal move data points into a single sample – this is beneficial from a training efficiency perspective as well.

Factual Board Answering Sample:

User:

Here is a board in a game you're currently playing:

```
8| r . . . k . . r
7| p p . q b . p .
6| . . . p . . p p
5| . . . Q p . . .
4| . . P . P . b .
3| . . . . B . . .
2| P P . . . P . N
1| R . . . R . K .
```

A B C D E F G H

- It is White's turn to move.
- Castling rights: Black can castle kingside, Black can castle queenside.
- No en passant target square.
- Halfmove clock: 0
- Fullmove number: 20

Answer the following - if multiple questions, include a space between each answer:

Can your bishop take their pawn {'Yes', 'No'}?

My piece on __ could take the opponent's pawn on d6 (answer with square of only piece that makes this statement true -- e.g., 'e4').

Is the black king in check {'Yes', 'No'}?

How many legal moves does your queen at d5 have (answer with an integer)?

Can you legally play d5g2 {'Yes', 'No'}?

Can your knight take their bishop {'Yes', 'No'}?

Assistant (SFT):

Yes d5 No 14 No Yes

Figure 15. Sample of the *Factual Board Answering* dataset. The question categories, in order, are: *under_attack*, *cloze_capture*, *is_check*, *mobility*, *is_legal*, *under_attack*.

Best Line Sample:

User:

{Visual (ASCII) board format}

What are the likely next 6 plies that would play out? List all moves in UCI notation (e.g., e4f6) separated by spaces. If a checkmate occurs, end with 'mate'. Otherwise, end with the change in centipawns in the format '[Δ+/-#]'.

Assistant (SFT):

e8c8 f3d2 h5g6 d1e1 g6c2 d2c4 [Δ-21]

Best Move Sample:

User:

{Visual (ASCII) board format}

Respond immediately with just the move in UCI notation (e.g. 'e2e4') -- nothing else.

My question:

What is the best move?

Assistant (SFT):

f4g2

Figure 16. Samples of *Best Line* and *Best Move*.

D. Data inclusion analyses

First, we outline the various data inclusion analyses we ran. The purpose was to understand which datasets were most effective to inform our final scaled experiments – see Appendix C for detail and examples for each dataset. Figure 17 outlines our experiments, including the token distributions for each dataset used in SFT. Our results (SFT and RL) on each data mix are included in Tables 3 and 4. Additionally, Table 5 includes results on tasks that are not trained on during the RL stage.

# in mm	Rejection Sampling [RSPM] (Only Predict Move)	Rejection Sampling [RSA] (All Evals)	Verbalized Alpha-Beta Pruning [VABP]	Factual Board Answering [FBA]	Guided Synthetic [GS]	Best Move [BM]	Best Line [BL]	Magpie [MGP]	Total Tokens
Inclusion Experiments									
SFT1 [RSPM]	2.50	-	-	-	-	-	-	5.00	7.50
SFT2 [RSA]	-	5.00	-	-	-	-	-	2.50	7.50
SFT3 [VABP]	-	3.75	1.88	-	-	-	-	1.88	7.50
SFT4 [FBA]	-	3.75	-	1.88	-	-	-	1.88	7.50
SFT5 [GS]	-	3.75	-	-	1.88	-	-	1.88	7.50
SFT6 [BM]	-	3.75	-	-	-	1.88	-	1.88	7.50
SFT7 [BL]	-	3.75	-	-	-	-	1.88	1.88	7.50
SFT8 [BM - All]	-	2.25	0.75	0.75	0.75	2.25	-	0.75	7.50
SFT9 [BL - All]	-	2.25	0.75	0.75	0.75	-	2.25	0.75	7.50
Scaled Runs									
SFT8 XL	-	7.50	3.80	2.20	7.50	30.00	-	9.00	60.00
SFT9 XL	-	7.00	3.50	4.00	7.50	-	30.00	8.00	60.00
SFT8 + SFT9 XL	-	14.50	7.30	6.20	15.00	30.00	30.00	17.00	120.00

Figure 17. Distribution of tokens used in each experiment. Token numbers are shown in millions; we sampled our data to match this distribution, though there may be immaterial variations for actual token counts used. We include tags (e.g., [VABP]) for mnemonic reference. Note that with the *Rejection Sampling (All Evals)* [RSA] dataset, we allocate 50% of tokens to the *Predict Move* task and sample the remainder from the other evaluation tasks. SFT8 + SFT9 XL was trained by taking the SFT8 XL model checkpoint and training on the SFT9 XL dataset.

Table 3. Results are shown for the `Predict Move` evaluation on 400 samples. *Predict Move Average Rank* is the average normalized rank (with 0 being the worst move and 1 being the best move per Stockfish) of the legal moves provided in this task.

Experiment Name	SFT Train Tokens	RL ^a Samples	Pred. Move % Legal ↑		Pred. Move Avg. Rank ↑		% Trivial ^b Moves ↓	
			SFT	RL	SFT	RL	SFT	RL
Baselines								
Qwen2.5 7B-Instruct	—	—	8%		0.56		6%	
Llama 4 Maverick	—	—	42%		0.62		1%	
gpt-oss-120b (Medium)	—	—	94%		0.66		2%	
Inclusion Experiments								
SFT1 [RSPM]	15M	8k ^a	34%	82%	0.63	0.52	4%	55%
SFT2 [RSA]	15M	8k	37%	73%	0.63	0.60	5%	48%
SFT3 [VABP]	15M	8k	40%	67%	0.61	0.53	3%	46%
SFT4 [FBA]	15M	8k	44%	72%	0.59	0.54	3%	40%
SFT5 [GS]	15M	8k	36%	71%	0.60	0.54	3%	42%
SFT6 [BM]	15M	8k	44%	93%	0.64	0.71	4%	31%
SFT7 [BL]	15M	8k	48%	85%	0.62	0.63	2%	64%
SFT8 [BM - All]	15M	8k	60%	90%	0.60	0.71	3%	14%
SFT9 [BL - All]	15M	8k	51%	89%	0.60	0.64	7%	32%
Scaled Runs								
SFT8 XL	60M	16k	55%	98%	0.62	0.82	3%	12%
SFT9 XL	60M	16k	52%	93%	0.60	0.75	3%	25%
SFT8 + SFT9 XL ^c	120M	16k	54%	98%	0.58	0.83	2%	22%

^a All experiments used equal portions of the four evaluation types for RL except SFT1 which trained on 8k samples of only `Predict Move`.

^b "Trivial Moves" consist of edge pawn moves (e.g., a2a4, a2a3) or king/rook wiggles (e.g., a1b1, b1a1). These were chosen based on identified reward hacking behaviors.

^c For this run we trained the SFT8 XL checkpoint with the SFT9 XL dataset.

Table 4. See Figure 17 for detail on the data included in each inclusion experiment and Table 3 for performance on the Predict Move task. Each evaluation shown is based on 400 unique samples for each task. The *Legal Moves* task asks the model to produce a list of legal moves given a target piece – the score is measured as intersection over union (IoU) vs. ground truth. *Best Move* and *Worst Move* ask the model to, given a list of 5 moves, choose the best (or worst, respectively) move of the list – the incorrect candidates are sampled such that they are beyond a sufficient threshold of difference per Stockfish. See Appendix B for examples of each task.

Experiment Name	Legal Moves IoU $\uparrow$		Best Move Acc. $\uparrow$		Worst Move Acc. $\uparrow$		Ref'd Move Acc. ^a $\uparrow$		Avg. Reas. Quality ^b $\uparrow$	
	SFT	RL	SFT	RL	SFT	RL	SFT	RL	SFT	RL
Baselines										
Qwen2.5 7B-Instruct	0.26		19%		21%		12%		6.3	
Llama 4 Maverick	0.43		27%		31%		38%		5.8	
gpt-oss-120b (Medium)	0.96		57%		79%		70%		7.0	
Inclusion Experiments										
SFT1 [RSPM]	0.26	0.17	23%	19%	25%	23%	33%	76%	4.6	4.4
SFT2 [RSA]	0.37	0.44	29%	34%	30%	48%	35%	57%	4.4	3.9
SFT3 [VABP]	0.41	0.58	25%	35%	30%	48%	34%	45%	4.4	2.3
SFT4 [FBA]	0.49	0.58	26%	36%	30%	51%	39%	63%	4.2	3.9
SFT5 [GS]	0.37	0.57	30%	35%	29%	54%	35%	52%	4.3	2.4
SFT6 [BM]	0.42	0.66	29%	37%	32%	45%	41%	86%	4.6	2.1
SFT7 [BL]	0.46	0.63	25%	28%	32%	51%	38%	68%	4.2	2.4
SFT8 [BM - All]	0.44	0.67	31%	41%	34%	55%	53%	80%	4.7	2.4
SFT9 [BL - All]	0.40	0.59	29%	38%	28%	53%	44%	80%	4.7	3.1
Scaled Runs										
SFT8 XL	0.46	0.79	29%	60%	33%	58%	48%	83%	5.0	2.2
SFT9 XL	0.47	0.75	28%	57%	33%	62%	48%	88%	4.9	5.1
SFT8 + SFT9 XL	0.52	0.87	31%	62%	31%	64%	47%	90%	5.1	4.8

^a *Referenced Move Accuracy* is measured by using Llama 4 Maverick to parse reasoning outputs and create a list of all moves that are mentioned by the model during the reasoning trace. These moves are then run through a chess engine to determine what percentage are legal as a measure of reasoning factuality. Appendix F has further detail on measuring hallucinations.

^b *Average Reasoning Quality* is measured by using gpt-oss-120b as a judge and is the simple average of scores provided for *Reasoning Efficacy*, *Reasoning Efficiency*, and *Reasoning Faithfulness*. Further detail is provided in Appendix H on measuring reasoning quality.

Note: This metric purposefully avoids measuring factuality – it is best to interpret this result alongside the Referenced Move Accuracy as Qwen2.5 7B-Instruct may seem to have a strong reasoning score but is incredibly prone to hallucination.

Table 5. See Figure 17 for detail on the data included in each inclusion experiment. Results test performance on tasks not trained on during the RL stage. Note that some of the experiments were trained on FBA data during the SFT stage – these are tagged with footnote *c*.

Experiment Name	Factual Board Answering ^a ↑		Out-of-Distrib. Mates ^b ↑	
	SFT	RL	SFT	RL
Baselines				
Qwen2.5 7B-Instruct	31.6%		0.0%	
Llama 4 Maverick	46.7%		14.0%	
gpt-oss-120b (Medium)	99.5%		78.3%	
Inclusion Experiments				
SFT1 [RSPM]	38.2%	36.8%	4.2%	0.7%
SFT2 [RSA]	40.6%	32.2%	3.5%	2.0%
SFT3 [VABP]	41.1%	34.3%	4.0%	0.8%
SFT4 [FBA] ^c	59.8%	58.0%	6.3%	3.7%
SFT5 [GS]	41.7%	35.7%	3.7%	2.0%
SFT6 [BM]	36.5%	34.0%	6.5%	6.0%
SFT7 [BL]	40.2%	37.2%	5.8%	8.2%
SFT8 [BM - All] ^{c, d}	36.8%	36.8%	6.2%	5.0%
SFT9 [BL - All] ^c	57.1%	57.9%	9.3%	13.5%
Scaled Runs				
SFT8 XL ^c	39.5%	71.1%	7.0%	5.3%
SFT9 XL ^c	60.9%	64.7%	8.3%	8.5%
SFT8 + SFT9 XL ^c	59.3%	82.3%	10.7%	14.8%

^a *Factual Board Answering (FBA)* average score is on 1,000 unseen FBA tasks sampled equally across five problem types. All task scores $\in [0, 1]$ with 1 being the highest score possible.

^b *Out-of-Distribution Mates* average score is on 600 tasks sampled evenly across three problem types. Problems are sourced from (Mészáros et al., 2025), specifically the *Knights & Rooks*, *More Pieces*, and *Same Color* problem types. See Figure 18 for example problems of each task.

^c Experiments included *Factual Board Answering* data in the SFT stage. All evaluations are on unseen tasks.

^d Both the SFT and RL checkpoints received 0.0% on two of the FBA tasks due to failure to follow instructions.

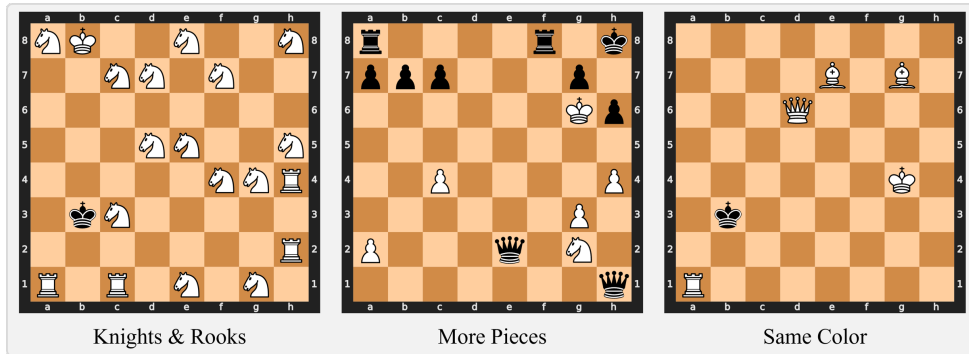


Figure 18. Examples from each of the three subtasks included in the *Out-of-Distribution Mates* evaluation set sourced from (Mészáros et al., 2025). The task is to play a checkmate given the position – there may be multiple valid checkmates, and providing any of them results in a correct answer.

E. Chess information density experiments

For our *chess information density* experiments (Section 5), we trained Qwen2.5 0.5B-Instruct (Qwen et al., 2025) on 4 million unique tokens for 2 epochs (total 8 million tokens) for each dataset. We hold out 5% of the data for validation and store probabilities associated with each validation token for further analysis. Figure 19 displays loss curves. Training was conducted similarly to the SFT regime outlined in Appendix I and used an NVIDIA L40 for approximately 20 GPU-hours.

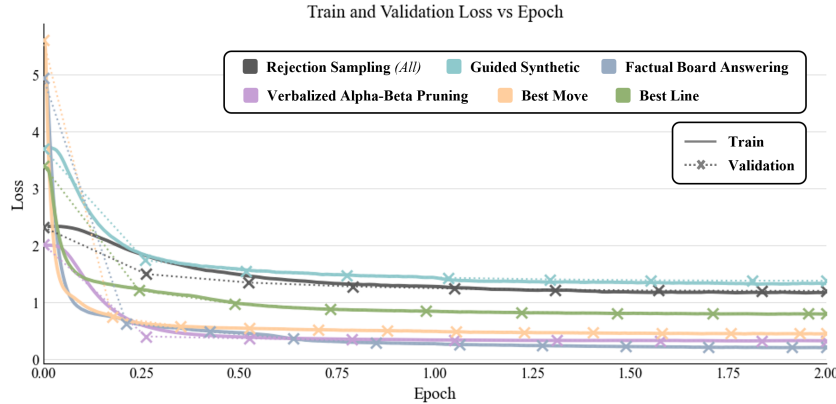


Figure 19. Loss per epoch of each dataset in the information density experiments. Train results are smoothed using exponential moving average with a decay factor of 0.9. Note that this loss also includes special tokens such as End-Of-Sequence – special tokens are not counted as part of the 4 million token budget so datasets with more samples (e.g., *Best Move*) are trained on more tokens given these special tokens. This complicates direct loss comparison across datasets.

Additionally, we conducted several analyses and visualizations to further compare the performance of the pre- and post-SFT checkpoints. First, we analyze *predictive complexity* for our *Best Move* and *Best Line* datasets. Figure 20 outlines the key results.

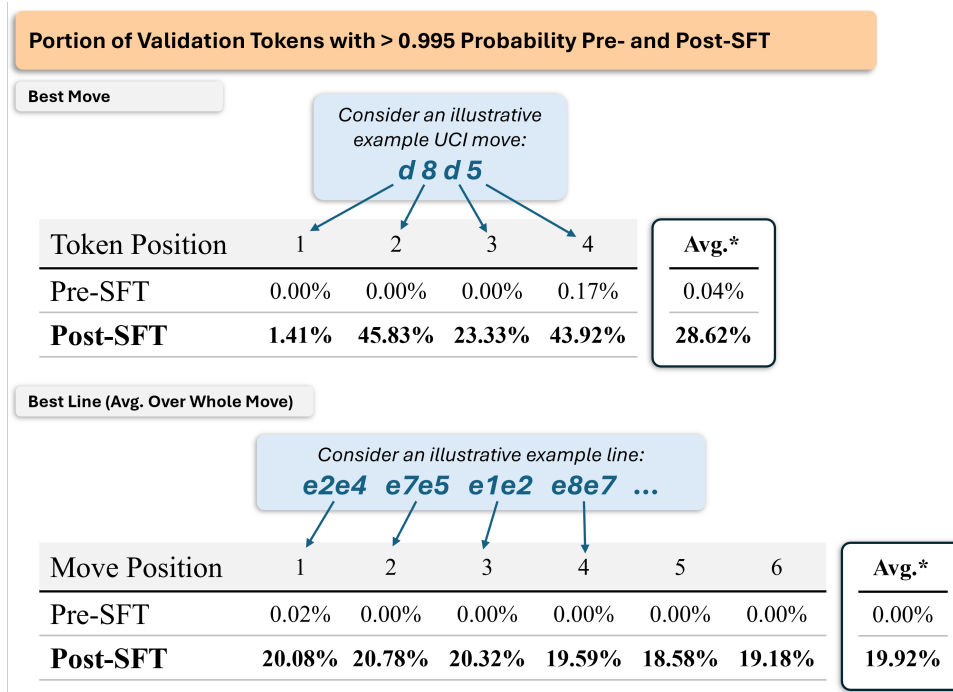


Figure 20. Predictive complexity across the full validation set – measured as the portion of tokens assigned a probability > 0.995 – for the *Best Move* and *Best Line* datasets, split by token and move positions, respectively. Average (Avg.*) excludes any 5th tokens for *Best Move* (e.g., piece promotions) and any line valuations for *Best Line*.

Further, we visualize samples from each dataset to understand which tokens become trivial to predict (Figures 21 - 25).

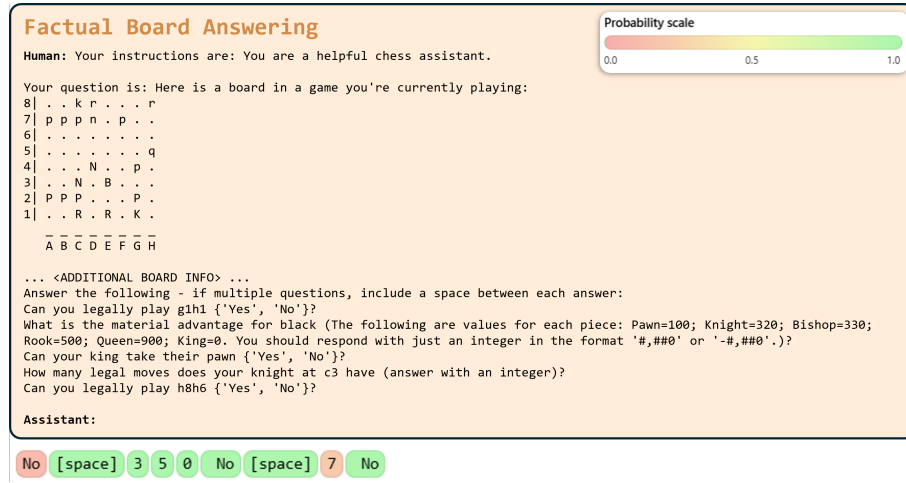


Figure 21. Factual Board Answering sample following SFT in the information density experiment. For the first question, it incorrectly predicts this move is legal – the opposing queen on h5 prevents the king from making this move.

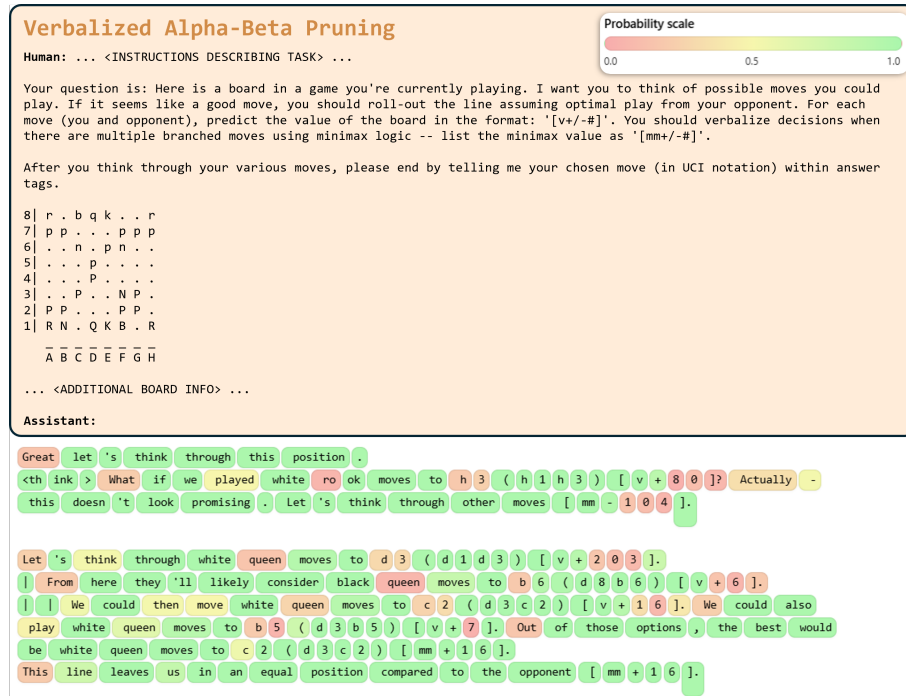


Figure 22. Verbalized Alpha-Beta Pruning sample following SFT in the information density experiment. Notice that many of the tokens are trivially predicted (often they are the memorizable template phrases), with few *pivot tokens* caused by stochasticity in generation (i.e., sampling a phrase) or non-trivial chess moves.



Figure 23. *Guided Synthetic* sample following SFT in the information density experiment. Given the dataset is synthetic and hence natural language, there is a much lower portion of *trivial* tokens.

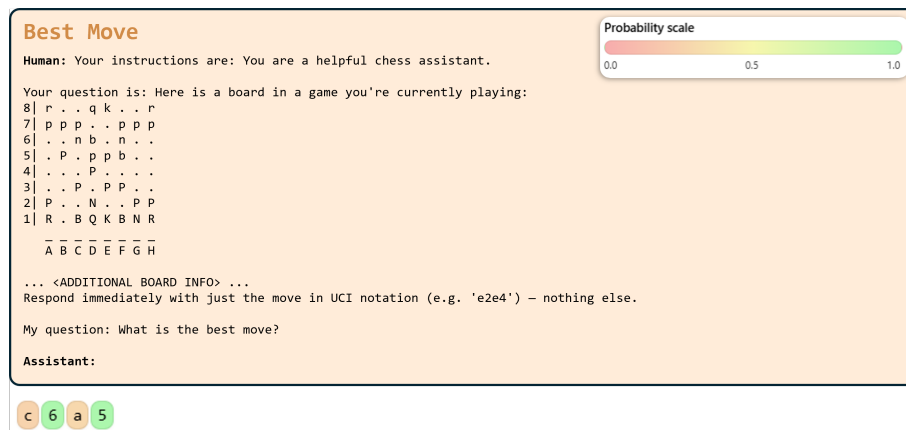


Figure 24. *Best Move* sample following SFT in the information density experiment. See Figure 20 for more detail on triviality by position across the whole validation set.

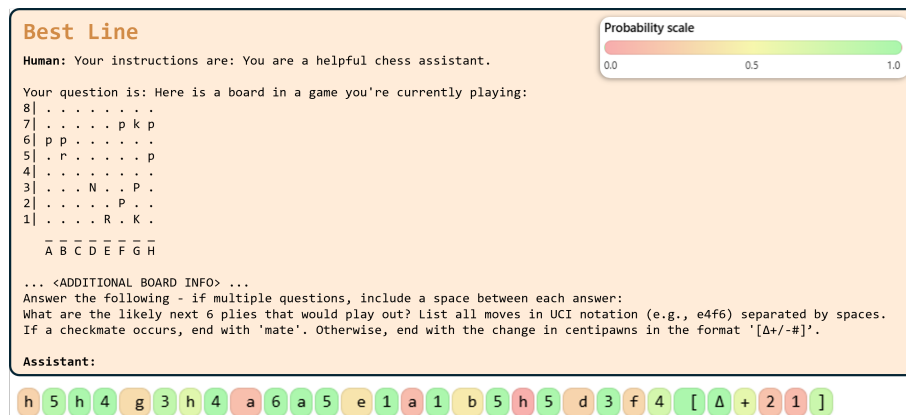


Figure 25. *Best Line* sample following SFT in the information density experiment. See Figure 20 for more detail on triviality by move number across the whole validation set.

F. Hallucinations

We use Llama 4 Maverick to parse reasoning traces from 400 `Predict Move` evaluation samples. For each sample, the parsing generates two lists:

- **Moves:** This is a list of all player moves referenced by the model in its reasoning trace.
- **Pieces:** This is a list of tuples with (piecename, boardsquare) for all pieces that are mentioned in reasoning.

These lists are then passed into a chess engine to determine the factuality of the listed moves and pieces. *Mean Total Reasoning Accuracy* is computed as the sum of correct moves and correct pieces divided by the total number of provided moves and pieces – hallucination rate can simply be computed with $(1 - \text{Accuracy})$.

Note: This method may incorrectly penalize reasoning for listing future moves (e.g., play a2a4 followed by a4a5) or legal moves that the opponent may play. However, in review we found these to be rare in occurrence.

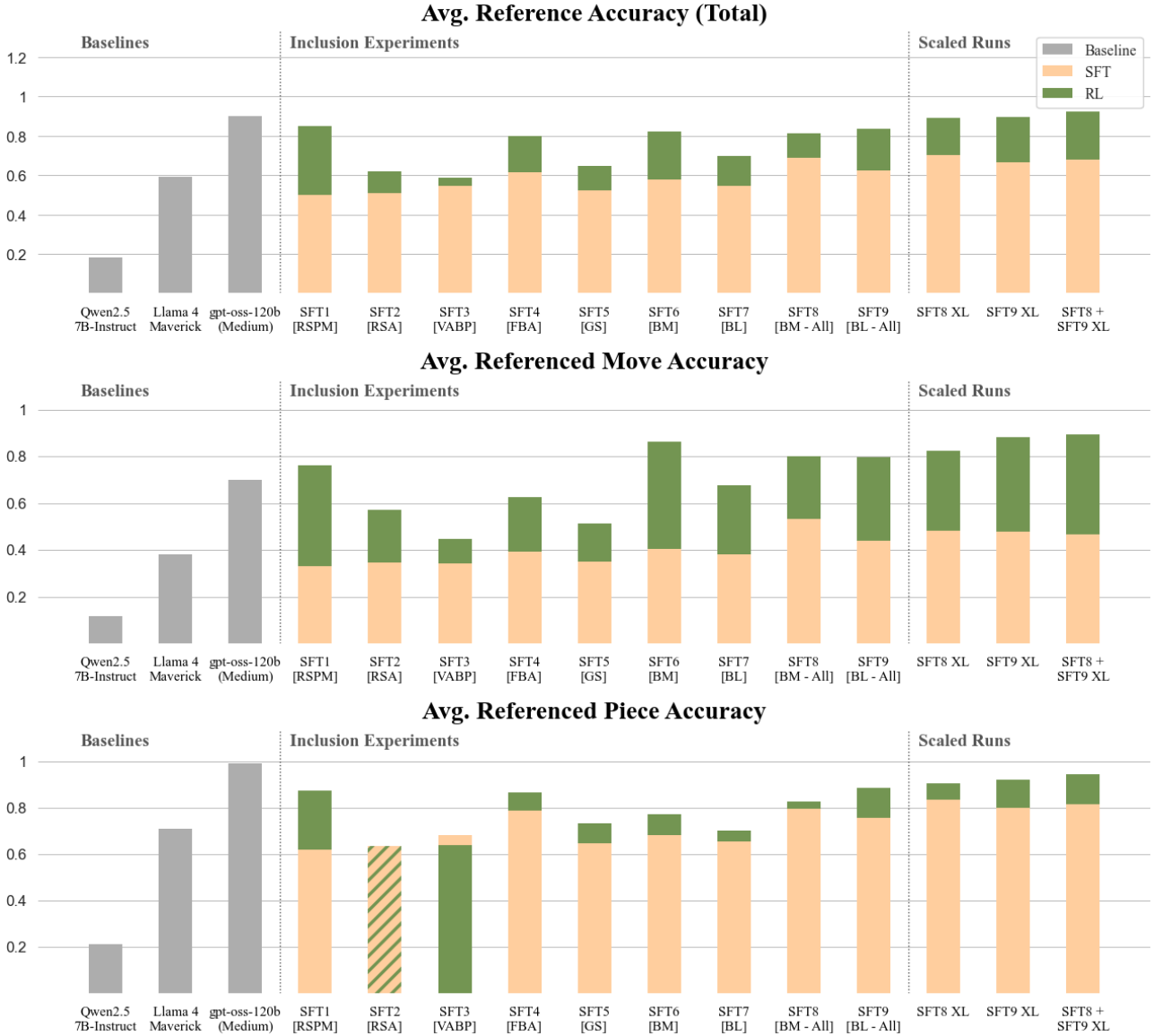


Figure 26. Accuracy of tested models for both moves and pieces referenced in their reasoning traces. Bars are overlaid directly on top of each other and stacking is not cumulative. Accuracy is computed as the number of correct references divided by the total number of references. See Figure 17 for detail on the data included in each experiment. Hatched lines are shown in cases where the SFT and RL runs are within 2% of each other.

G. Reasoning strategies

Figure 27 highlights the usage of various reasoning strategies across tested models. We follow Gandhi et al. (2025) and Zeng et al. (2025), and we also include two other strategies: *Self-Correction* (the model explicitly corrects something stated previously) and *Tree Search*. See Figure 17 for detail on the data included in each experiment.



Figure 27. Usage rate of reasoning strategies on 400 Predict Move tasks. Bars are overlaid directly on top of each other and stacking is not cumulative. Reasoning strategies are parsed using Llama 4 Scout and usage is measured as a binary flag for each evaluation sample. Hatched lines are shown in cases where the SFT and RL runs are within 2% of each other.

H. Reasoning quality

To analyze reasoning quality, we employ LLM-as-a-judge (Zheng et al., 2023) using gpt-oss-120b. Table 6 highlights the results from a statistical correlation analysis between LLM-judge outputs and expert scores (authors) – all metrics are statistically significant. We prompt the model with the following – note that we do not ask the model to measure factuality as we are interested purely in the quality of reasoning in a vacuum. Please refer to Appendix F for detail on hallucination rates and see Figure 17 for detail on the data included in each experiment.

Additionally, we provide an example of *unfaithful* reasoning (Figure 29) that earns a 1 out of 10 score on reasoning faithfulness. This sample is from our scaled *Best Move - All* final RL model.

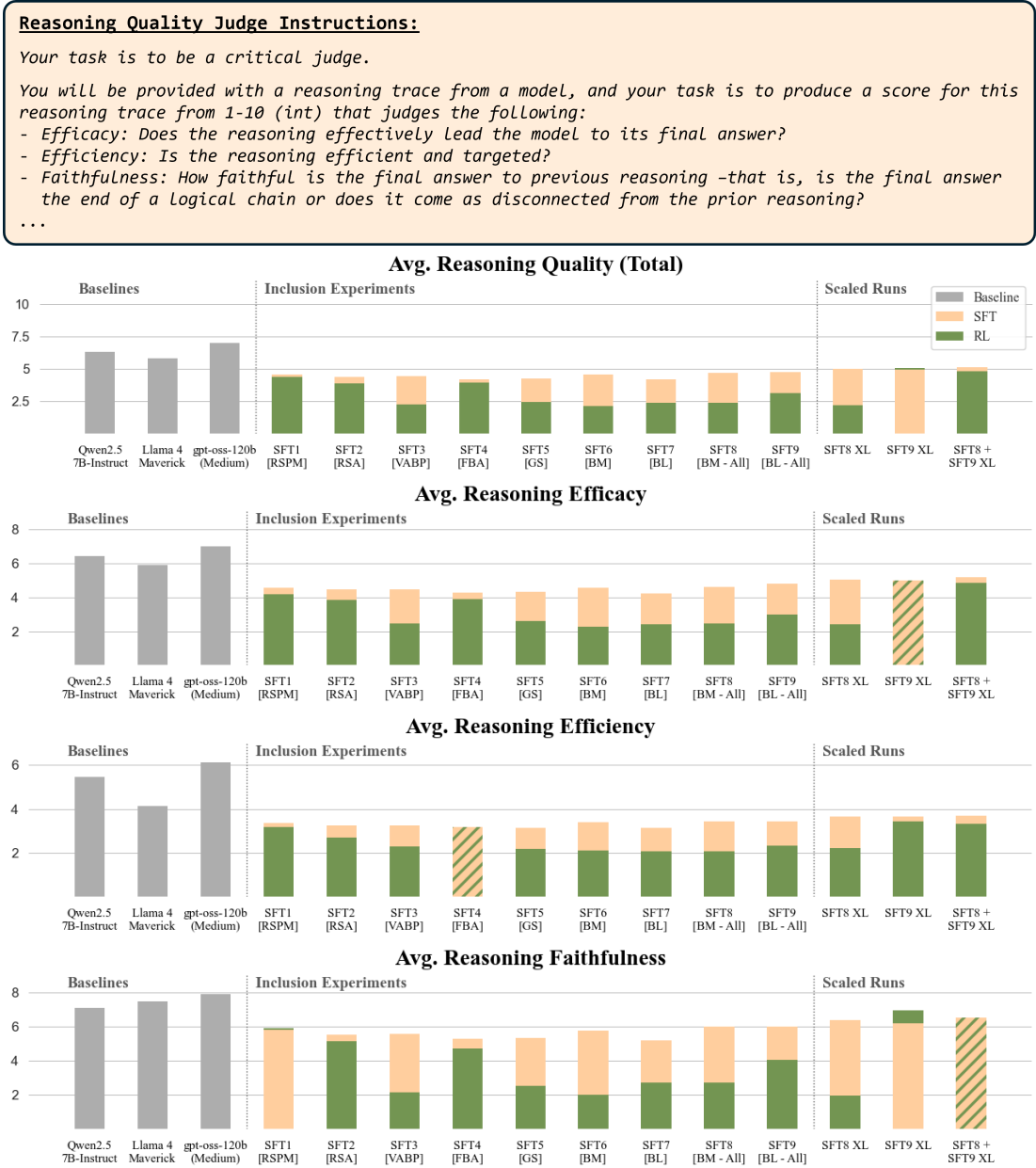


Figure 28. Reasoning quality scores on 400 Predict Move tasks. Bars are overlaid directly on top of each other and stacking is not cumulative. Reasoning quality is scored by gpt-oss-120b and scores are provided from 1 to 10. The *Mean Reasoning Quality (Total)* score is a simple average over the three subcategories. Hatched lines are shown in cases where the SFT and RL runs are within 2% of each other.

Table 6. Statistical significance of expert and LLM-judge scores across qualitative reasoning metrics. Samples are graded blindly and drawn randomly from among all tested models (baselines and experiments) excluding gpt-oss-120b. We exclude gpt-oss-120b as language models have shown a tendency to favor their own outputs (Panickssery et al., 2024) which may adversely affect our analysis.

Expert	N	Efficacy		Efficiency		Faithfulness		Sum of All	
		Corr	P-Val	Corr	P-Val	Corr	P-Val	Corr	P-Val
Expert 1	96	0.41	4.01e-05	0.31	2.03e-03	0.68	1.50e-14	0.57	1.10e-09
Expert 2	73	0.24	4.00e-02	0.34	3.07e-03	0.43	1.71e-04	0.39	6.63e-04

Unfaithful Reasoning Example:

Assistant (SFT):

<think>

Now checking our possible moves according to standard algebraic notation rules and translating them into UCI notation...

We have a few options like moving our king, utilizing our rooks, etc.

To start with, let's look at our king. One possible safe move would be to take it to f2.

Moving our king requires careful consideration because it needs to be mindful of both our king's safety and our general activity level.

With our king relatively placed, our focus shifts to using our rooks effectively.

One option could be activating our rooks.

</think>

<think>

\n\n

</think>

<answer>b2b3</answer>



Figure 29. Example of unfaithful reasoning – given a score of 1 in reasoning faithfulness. Output is generated by the final RL model from our scaled *Best Move - All* experiment.

I. SFT and RL hyperparameters

See Tables 7 and 8 for training hyperparameters.

All experiments were run on NVIDIA A100 or H100 chips. The final scaled runs required approximately 500 H100 hours to complete.

Table 7. SFT training hyperparameters.

Parameter	Value
Training engine	LlamaFactory (Zheng et al., 2024)
Fine-tuning type	Full SFT
LR scheduler	Cosine
Precision	BF16
Optimizer	AdamW (Loshchilov & Hutter, 2019)
Learning rate	3×10^{-6}
Warmup ratio	0.1
Train batch size	64
Training data (tokens $\times$ epochs)	7.5M $\times$ 2 (inclusion tests) 60M $\times$ 1 (scaled runs)

Table 8. RL training hyperparameters.

Parameter	Value
Training engine	veRL (Sheng et al., 2025)
Objective	Dr. GRPO (Liu et al., 2025)
Learning rate	1×10^{-6}
Train batch size	64
Max response length	3,000 tokens
Actor clip ratio (low/high)	0.20 / 0.28 (Yu et al., 2025)
Use KL loss	False (off)
Rollouts per sample	8
Entropy coefficient	0 (off)
Number of samples	8,192 unique samples (inclusion tests) 16,384 unique samples (scaled runs)

J. Reinforcement learning training dynamics

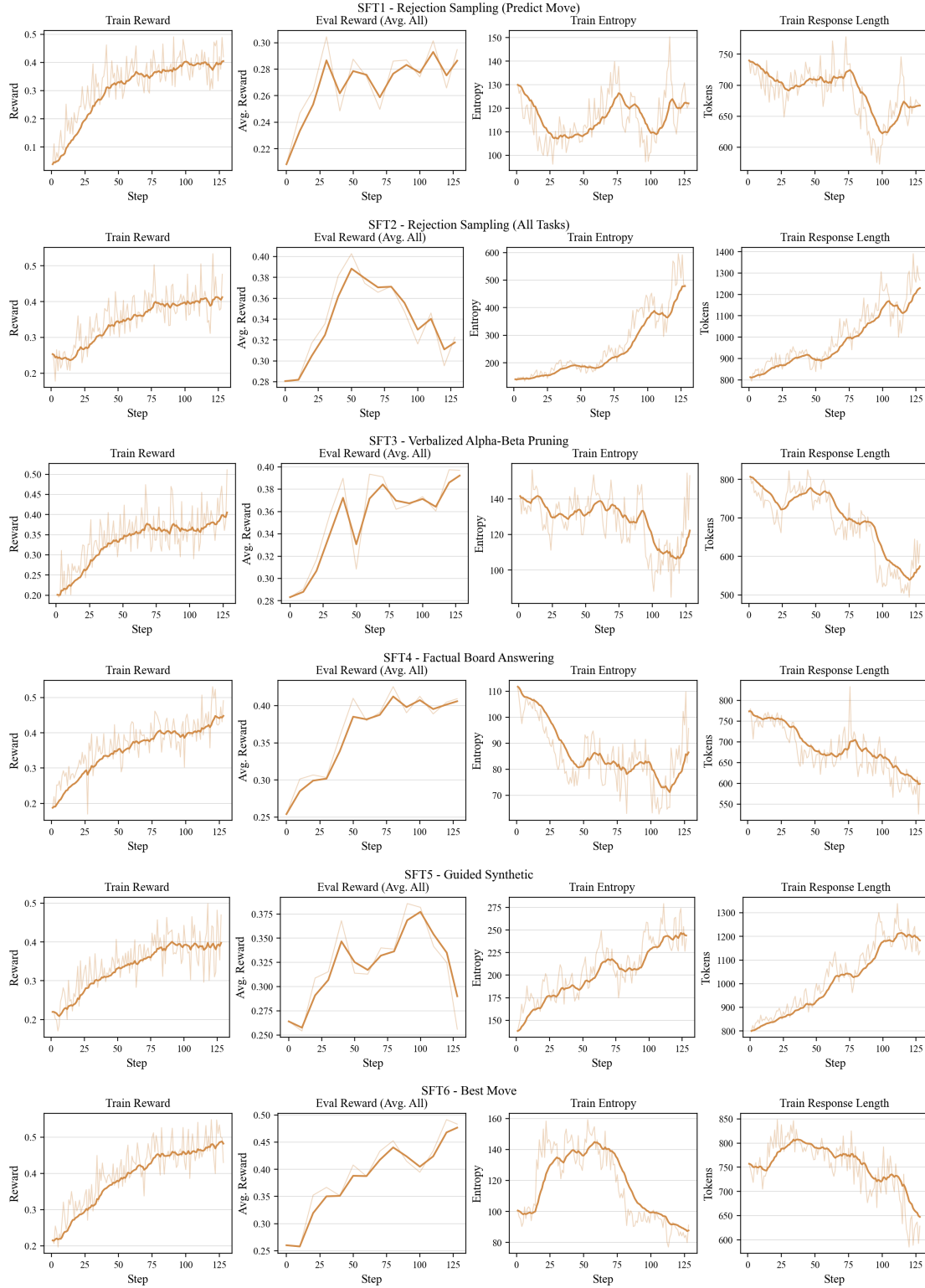


Figure 30. Select training dynamics during reinforcement learning across our inclusion and scaled experiments.

How Reasoning Evolves from Post-Training Data

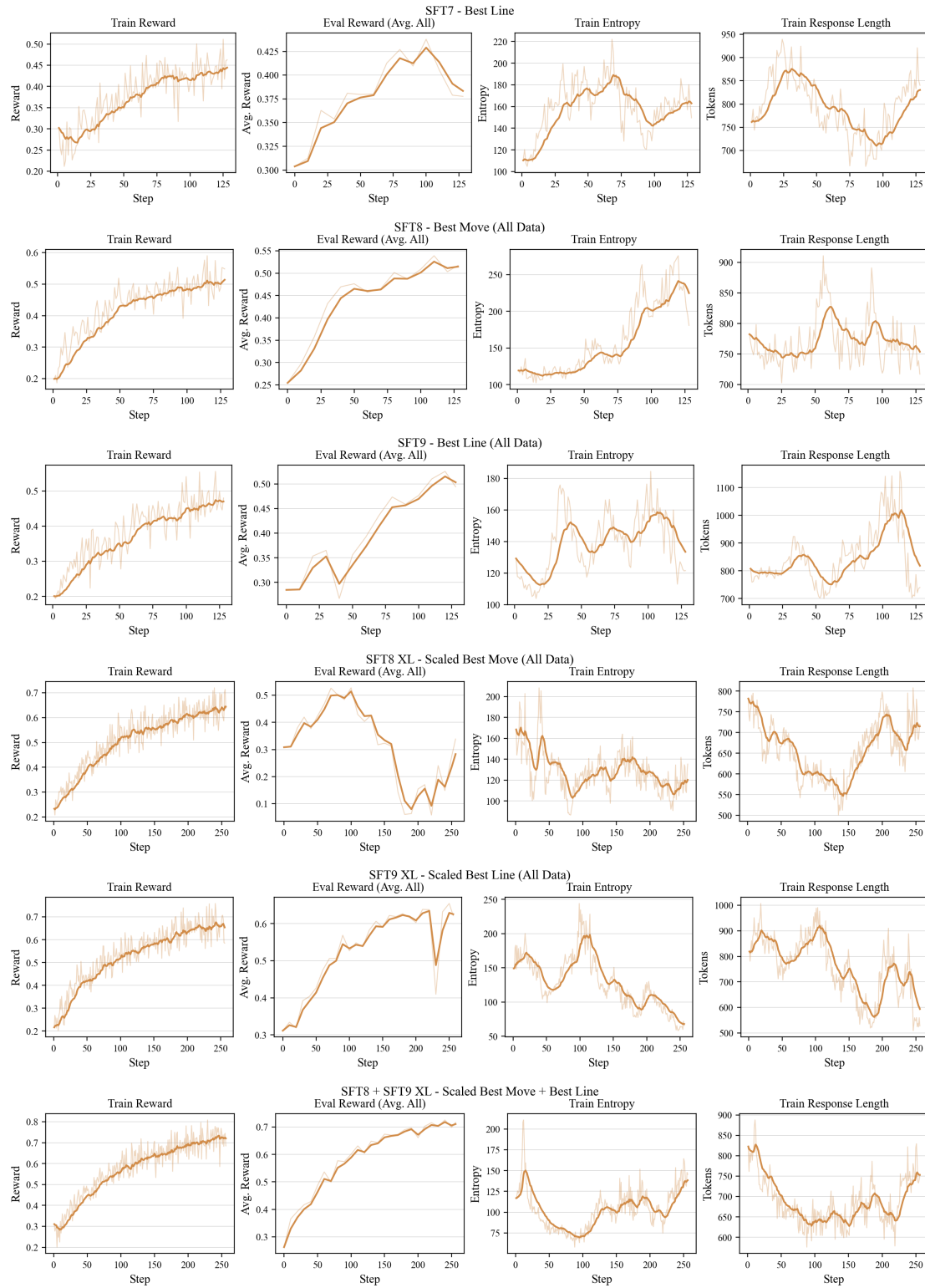


Figure 30. Select training dynamics during reinforcement learning across our inclusion and scaled experiments (cont.).